

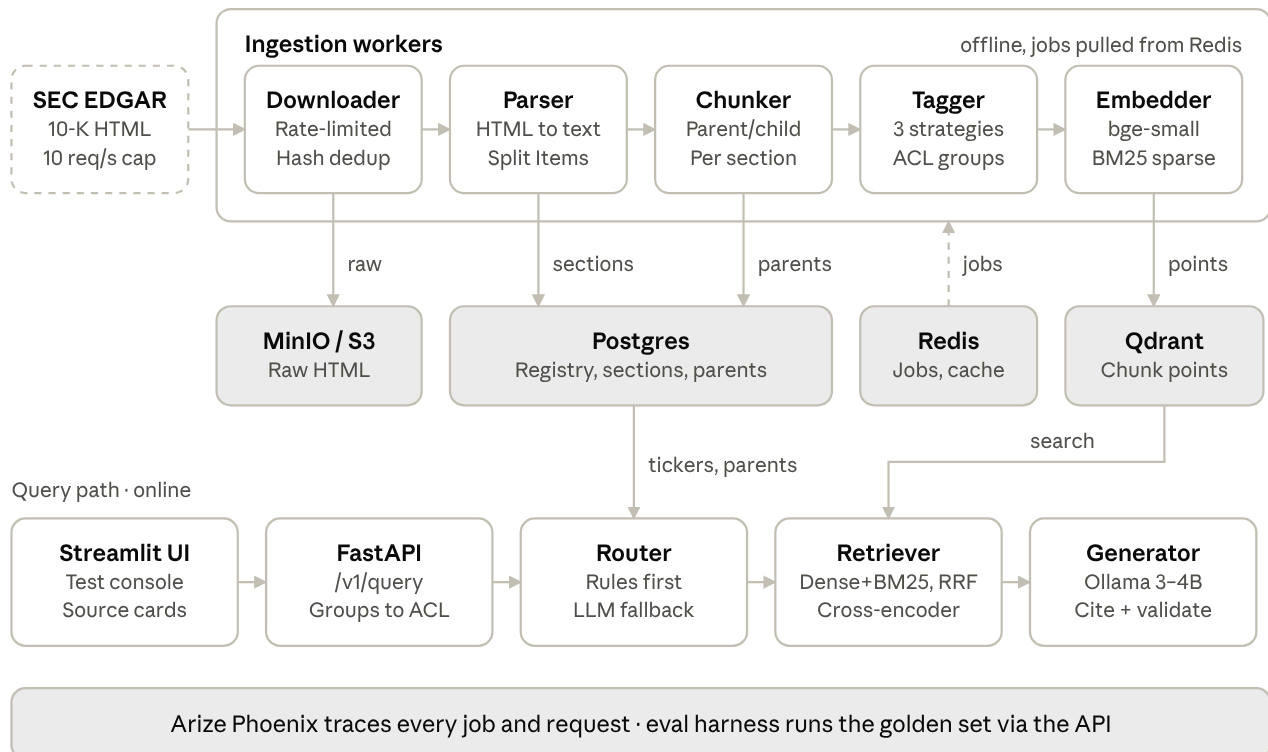
SEC 10-K Analyst Assistant — Architecture and Engineering Plan

Oct 2, 2026 · @Atul kumar

The system is a set of offline ingestion workers and an online query API sharing four stores, built locally with Docker Compose on a CPU-only laptop and deployed to AWS at the end; development runs in nine engineering steps over about 24 weekends.

Architecture at a glance

Offline workers fill the stores; the online query path reads them



system block diagram · ingestion workers, stores, query path

The two paths never call each other: workers write to the stores, and the API reads company names and parent passages from Postgres and chunk points from Qdrant. Dashed boxes are external; dashed arrows are queue pulls.

Components and responsibilities

The system has eleven components in three groups; each has one job and talks to the others only through a store, a queue or an HTTP call.

Group	Component	Responsibility	Runs as
Ingestion	Downloader	Resolve ticker → CIK, list 10-K filings, download primary HTML within the SEC rate limit, hash and store raw files	RQ worker
Ingestion	Parser	Convert HTML to clean text, split into Items (1, 1A, 1C, 3, 7, 7A), keep tables intact	RQ worker
Ingestion	Chunker	Build parent passages and child chunks per section, count tokens	RQ worker
Ingestion	Tagger	Attach metadata, run rules / zero-shot / clustering categorization, assign access groups	RQ worker (clustering as a batch job)
Ingestion	Embedder	Create dense and sparse vectors, upsert Qdrant points, skip unchanged chunks by hash	RQ worker
Query	API	Validate requests, resolve the caller's groups, orchestrate the query pipeline, stream responses	FastAPI service
Query	Router	Extract tickers, years, sections and intent from the question	Library inside the API
Query	Retriever	Hybrid search with filters, Reciprocal Rank Fusion, cross-encoder rerank, parent expansion	Library inside the API
Query	Generator	Prompt the local LLM with numbered context, validate citations, set <code>status</code>	Library inside the API + Ollama
Support	Eval harness	Run the golden set against the API and store metrics per configuration	CLI (<code>make eval</code>)

Group	Component	Responsibility	Runs as
Support	Tracing	Record per-stage latency and inputs/outputs for every request and job	Arize Phoenix container

The router, retriever and generator are libraries rather than separate services: at this scale extra network hops would add latency and operational work without any benefit.

Tech stack

Every choice favours a CPU-only 16 GB laptop and keeps one named alternative to benchmark, which is where the architecture-decision records (ADRs) come from.

Layer	Choice	Why	Alternative to compare
Language and tooling	Python 3.12, uv, ruff, mypy, pytest, pre-commit	Fast installs, strict typing, one-command checks	Poetry
EDGAR client	httpx + tenacity (retries) + a token-bucket limiter	Full control over User-Agent and the 10 req/s ceiling; good learning	edgartools library
HTML parsing	selectolax / BeautifulSoup + lxml, regex Item splitter	Light on CPU; Item headings are predictable	sec-api.io extractor (paid)
Schemas and config	Pydantic v2, pydantic-settings	One typed model for documents, chunks, API contracts	dataclasses
Relational store	PostgreSQL 16, SQLAlchemy 2.0, Alembic migrations	Registry, state machine, parent chunks, eval runs	SQLite
Queue	Redis + RQ	Simple workers, retries, failed-job registry	Celery, arq
Raw file store	MinIO locally, S3 on AWS (boto3)	Same code path in both environments	Local disk

Layer	Choice	Why	Alternative to compare
Dense embeddings	FastEmbed (ONNX) with bge-small-en-v1.5, 384 dims	Fast on CPU, small index	e5-small-v2, nomic-embed-text
Sparse vectors	BM25 via FastEmbed, stored as Qdrant sparse vectors	Exact matches for tickers and legal terms	SPLADE
Vector database	Qdrant	Named dense + sparse vectors, payload filters and indexes, collection aliases	pgvector
Reranker	Small cross-encoder (ms-marco-MiniLM-L-6-v2)	Big precision gain at low CPU cost	bge-reranker-base
Topic discovery	BERTopic (UMAP + HDBSCAN)	Finds risk themes the taxonomy missed	k-means
LLM	Ollama, 3–4B instruct model, Q4 quantized, JSON-schema output for routing	Runs on CPU, no API cost	7–8B model (slower)
API	FastAPI + Uvicorn	Async, typed, auto OpenAPI docs	Flask
Test UI	Streamlit	Zero frontend effort	—
Tracing	Arize Phoenix via OpenTelemetry	Single container, fits in RAM	Langfuse (hosted)
Evaluation	Own recall@k / MRR / nDCG + LLM-judged faithfulness	Retrieval and answer quality measured separately	RAGAS
Load testing	Locust	Python-native	k6
Packaging and CI	Docker Compose, GitHub Actions	Same stack everywhere; lint and tests on every push	—

Layer	Choice	Why	Alternative to compare
Cloud (final phase)	Terraform, S3, SQS, ECR, EC2	Create and destroy on demand	AWS CDK

Repository layout

One repository, one Python package, with folders that mirror the components so each can be tested in isolation.

```

sec-rag/
├─ src/secrag/
│   ├─ common/           # config, logging, tracing, pydantic models
│   ├─ edgar/            # EDGAR client, rate limiter, ticker → CIK
│   ├─ ingestion/        # downloader, parser, chunker, tagger, embedder jobs
│   ├─ categorize/       # rules, zero-shot, clustering
│   ├─ storage/          # Postgres repositories, Qdrant client, object store
│   └─ retrieval/        # router, hybrid search, fusion, rerank, parent
├─ expansion
│   ├─ generation/       # prompts, Ollama client, citation validator
│   ├─ api/              # FastAPI app, routes, dependencies
│   └─ eval/             # golden-set loader, metrics, runner
├─ ui/streamlit_app.py
├─ migrations/          # Alembic
├─ data/golden/          # golden questions (YAML)
├─ tests/                # unit/, integration/, fixtures/ (saved sample
filings)
├─ infra/
│   ├─ docker-compose.yml
│   └─ terraform/
├─ docs/adr/             # ADR-001 ... ADR-0NN
├─ Makefile              # up, down, test, ingest, eval, load-test
└─ pyproject.toml

```

Saved sample filings in `tests/fixtures/` let parser and chunker tests run offline, without touching EDGAR.

Ingestion pipeline design

Ingestion is a chain of small, idempotent jobs: each job reads one filing's state from Postgres, does one stage, writes its output and enqueues the next stage, so any job can be retried or re-run safely.

Filing state machine (column `filings.status`)

```
discovered → downloaded → parsed → chunked → tagged → embedded → indexed
    ↳ failed (with stage + error; retried, then dead-lettered)
```

Design rules

- **Idempotency:** the job key is `(accession_no, stage)`. A job whose output already exists for the current input hash exits as a no-op.
- **Dedup at three levels:** raw file SHA-256 (skip re-download), section text hash (skip re-chunking), chunk hash (skip re-embedding).
- **Retries:** network and SEC errors retry with exponential backoff (3 attempts); parse errors go straight to `failed` with the error stored, because retrying will not fix them.
- **Rate limiting:** one shared token bucket in Redis keeps every downloader worker together under the SEC ceiling (target 8 req/s for headroom).
- **Batching:** the embedder processes chunks in batches (size tuned by benchmark) and upserts to Qdrant in batches.
- **Re-indexing:** a new embedding model or chunking strategy writes to a new collection (`chunks_v2`); after evaluation passes, the alias `chunks` is switched, giving zero-downtime swaps and instant rollback.
- **Observability:** every job emits a trace span with filing id, stage, duration and outcome.

Batch entry point: `make ingest TICKERS=AAPL,MSFT YEARS=2023-2025` enqueues `discover` jobs; `make status` prints counts per state from Postgres.

Query pipeline design

Every request passes through nine steps; three of them can end the request early (cache hit, access denied, nothing relevant found).

1. **Cache check.** Key = normalized question + caller's groups. A hit returns the stored response from Redis. (Exact-match cache first; semantic cache is an E7 experiment.)

2. **Route.** Rules first: ticker and company-name dictionary from the `companies` table, year patterns ("FY2024", "last year"), section keywords ("risk" → 1A, "results" → 7). Only when rules find no company or the intent is unclear does the local LLM run, returning a JSON object that matches a Pydantic schema.
3. **Access control.** Caller's groups map to allowed sectors; a `must` filter on `allowed_groups` is added to every search. If every requested ticker is outside the caller's access, return `access_denied` before any search runs.
4. **Retrieve.** For each requested company separately (so one long filing cannot crowd out another): dense top 30 and sparse top 30 with filters.
5. **Fuse.** Reciprocal Rank Fusion merges the dense and sparse lists into the top 20 candidates.
6. **Rerank.** The cross-encoder scores the 20 candidates against the question; keep the top 5 (top 3 per company for comparisons). If the best score is below a tuned threshold, return `not_found`.
7. **Expand.** Replace each child with its parent passage from Postgres, de-duplicate parents, and trim to a fixed context budget.
8. **Generate.** Prompt the LLM with numbered sources, instructions to cite every claim as `[n]`, and to say so when the sources do not cover something.
9. **Validate and respond.** Every `[n]` must point to a supplied source; unsupported claims set `status = partial`. Build `citations`, `filters_applied` and `debug`, cache the response, close the trace.

The thresholds and top-k values above are starting points; E6 and E7 tune them on the golden set.

Interfaces and data contracts

Three contracts hold the system together: the public API, the job messages between workers, and the Qdrant collection schema. The full request and response bodies are in the project deep-dive doc.

API endpoints

Method	Path	Purpose
POST	<code>/v1/query</code>	Ask a question; returns answer, citations, status, filters, debug (optionally streamed)

Method	Path	Purpose
POST	/v1/admin/ingest	Enqueue ingestion for tickers and fiscal years
GET	/v1/filings	List filings and their pipeline status, filterable by ticker and status
GET	/v1/eval/runs	List evaluation runs and their metrics
GET	/v1/health	Liveness plus Postgres, Qdrant, Redis and Ollama checks

Job messages (RQ, one function per stage, payload = ids only, never large text)

Job	Input	Output	Next
discover_filings	ticker, fiscal years	filings rows in discovered	download_filing per filing
download_filing	accession_no	Raw HTML in object store, hash	parse_filing
parse_filing	accession_no	sections rows	chunk_filing
chunk_filing	accession_no, strategy	parent_chunks rows, child chunks staged	tag_filing
tag_filing	accession_no	Metadata and categories on child chunks	embed_filing
embed_filing	accession_no, model	Qdrant points upserted	Status → indexed
cluster_topics	collection	topic_cluster on all points	None (batch job)

Qdrant collection chunks_v1 (alias chunks)

```

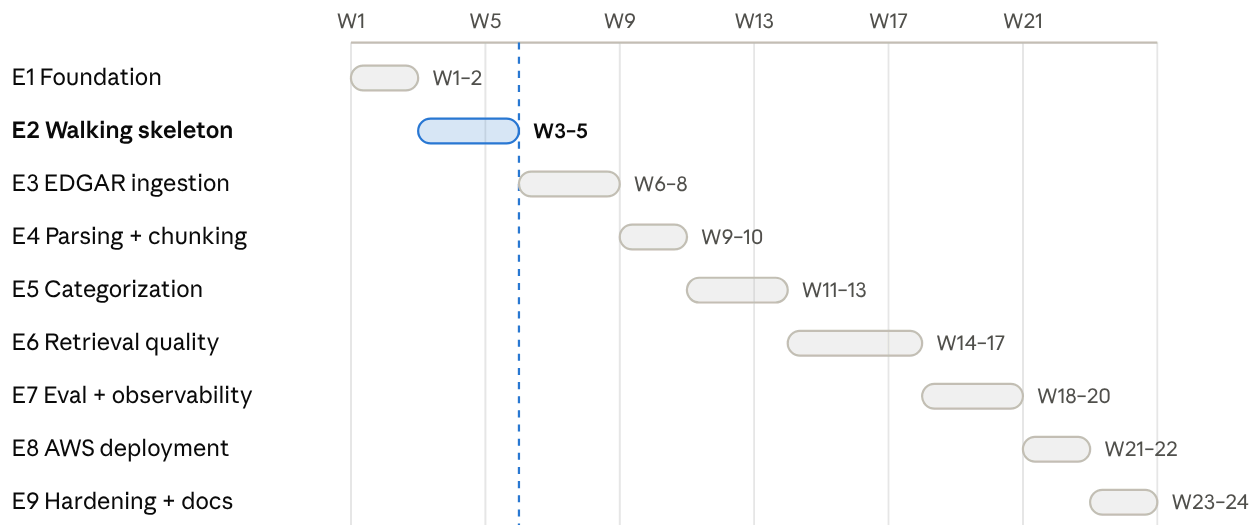
vectors:          dense → size 384, distance Cosine
sparse_vectors:   bm25  → IDF modifier
payload indexes:
  keyword:  ticker, section, sector, allowed_groups, risk_categories,
  topic_cluster
  integer:  fiscal_year
optional:  scalar int8 quantization (benchmarked in E6)

```

Engineering timeline

An end-to-end skeleton runs by week 5; seven steps then deepen it

Engineering steps by week at 5-8 hours per weekend



engineering timeline · 9 steps over 24 weeks

E2 deliberately ships a naive version of every layer first, so each later step has a baseline to beat on the golden set.

Engineering steps

Each step ends in something runnable and a definition of done (DoD) that can be checked; later steps deepen what E2 builds rather than adding new layers.

E1 — Foundation (weeks 1-2)

- ☐ Repository layout, uv project, ruff, mypy, pytest, pre-commit, Makefile

- ☐ Docker Compose: Postgres, Qdrant, Redis, MinIO, Phoenix, each with a health check; Ollama installed natively with the chosen model pulled
- ☐ pydantic-settings config, structured JSON logging, OpenTelemetry tracing wired to Phoenix
- ☐ Alembic set up with the first `companies` and `filings` tables
- ☐ GitHub Actions: lint, type-check, unit tests

DoD: `make up` starts everything healthy; CI is green. **ADR-001:** repository and service boundaries.

E2 — Walking skeleton (weeks 3–5)

- ☐ Minimal EDGAR client: ticker → CIK → latest 10-K for 5 companies, saved with hash
- ☐ Naive parse (whole document text), fixed-size chunks, bge-small dense vectors, one Qdrant collection
- ☐ `/v1/query`: dense top 5 → prompt → Ollama → answer with citations
- ☐ Streamlit page showing answer and source cards
- ☐ Golden set v1: 20 questions with expected filing and section; `make eval` reports recall@5, MRR, p50/p95 latency

DoD: baseline metrics saved and reproducible. **ADR-002:** baseline design and known weaknesses.

E3 — EDGAR ingestion at scale (weeks 6–8)

- ☐ Full job chain on RQ with the state machine, retries, failed-job handling
- ☐ Shared Redis token-bucket rate limiter and declared User-Agent
- ☐ Raw files in MinIO; dedup by SHA-256; `10-K/A` and fiscal-year edge cases handled
- ☐ `make ingest` and `make status`; full corpus (about 150 filings) downloaded

DoD: killing a worker mid-run and restarting finishes with no duplicates; re-running ingestion is a no-op. **ADR-003:** queue and retry semantics.

E4 — Section parsing and chunking (weeks 9–10)

- ☐ Item splitter for 1, 1A, 1C, 3, 7, 7A with fixture tests on at least 10 real filings from different filers
- ☐ Tables kept whole inside a chunk; boilerplate (page headers, table of contents) removed

- ☐ Parent passages in Postgres; child chunks sized by tokens; compare fixed vs heading-aware vs parent-child on the golden set
- ☐ Golden set grows to 60 questions covering all eight question types

DoD: section detection works on every corpus filing or the filing is flagged; chunking comparison table recorded. **ADR-004:** chunking strategy.

E5 — Metadata and categorization (weeks 11–13)

- ☐ Rules: sector from SIC code, section from parser, access groups from sector
- ☐ Zero-shot: risk taxonomy (about 8–10 labels with descriptions) assigned by embedding similarity, multi-label with a threshold
- ☐ Clustering: BERTopic over Item 1A chunks, review and name clusters, write `topic_cluster`
- ☐ Label 100 chunks by hand to measure precision of each method
- ☐ Payload indexes created; filtered-query latency measured before and after

DoD: every point carries complete metadata; a precision table exists for each categorization method. **ADR-005:** taxonomy and categorization approach.

E6 — Retrieval quality (weeks 14–17)

- ☐ Router: rules first, LLM fallback with JSON schema; router accuracy measured on the golden set
- ☐ Sparse BM25 vectors added; hybrid search with RRF; per-company retrieval for comparisons
- ☐ Cross-encoder rerank and confidence threshold for `not_found`
- ☐ Access-control filter plus tests proving no restricted chunk ever reaches the prompt
- ☐ Citation validator and `partial` status; streaming responses
- ☐ Embedding model comparison (bge-small vs e5-small) via a new collection and alias switch

DoD: an ablation table shows what routing, hybrid, rerank and parent expansion each add to recall@5 and latency. **ADR-006:** retrieval design. **ADR-007:** re-index strategy.

E7 — Evaluation, observability and performance (weeks 18–20)

- ☐ Synthetic questions generated overnight with Ollama, filtered, merged into the golden set

- ☐ Answer faithfulness and relevance scored by an LLM judge, checked against 20 manual labels
- ☐ Phoenix dashboards: latency per stage, slowest requests, failure reasons
- ☐ Response cache in Redis; semantic cache experiment
- ☐ Locust load test to find where p95 breaks and which stage causes it

DoD: one report with retrieval quality, answer quality and per-stage latency, bottleneck named with numbers. **ADR-008:** evaluation methodology.

E8 — AWS deployment (weeks 21–22)

- ☐ New AWS account: budget alerts, least-privilege IAM, region ap-south-1
- ☐ Terraform: S3, SQS with dead-letter queue, ECR, one EC2 instance, security groups, IAM roles
- ☐ Storage switched from MinIO to S3 by configuration only; S3 upload event → SQS → worker
- ☐ Docker images pushed to ECR and run on EC2 with Compose
- ☐ `terraform destroy` after each session, verified in the billing console

DoD: a filing dropped in S3 becomes searchable without manual steps; teardown leaves nothing running. **ADR-009:** deployment topology and cost.

E9 — Hardening and documentation (weeks 23–24)

- ☐ README with architecture diagram, setup, and results tables
- ☐ ADRs consolidated; one-page note on scaling to 10,000 companies and 20 years
- ☐ Five-minute demo recording; rehearsed answers to the likely interview questions

DoD: a new developer can clone, run `make up && make ingest && make eval`, and reproduce the reported numbers.

Quality and performance targets

These are starting targets for a CPU-only laptop, to be revised once the E2 baseline exists; the point is to have a number to beat, not to guess the final one.

Area	Metric	Starting target	Measured in
Retrieval quality	recall@5 on the golden set	Clearly above the E2 baseline; set the exact bar after E2	E2, E4, E6
Ranking quality	MRR	Improves with each E6 component, or the component is dropped	E6
Answer quality	Faithfulness (claims supported by cited sources)	No unsupported claim marked <code>answered</code>	E7
Access control	Restricted chunks in prompts	Zero, enforced by tests	E6
Latency (excluding generation)	p95 of route + retrieve + rerank	Under 1.5 s	E6, E7
Latency (end to end)	p95 with local 3–4B model	Measured and reported; generation is expected to dominate	E7
Ingestion	Filings per hour, per stage	Measured; full corpus re-runs as a no-op	E3
Reliability	Worker crash mid-run	No duplicates, no lost filings	E3

Decisions taken and open questions

Decided

- Local-first on a CPU-only 16 GB laptop; AWS only in E8, on a new account created at that time
- Open-source embeddings (bge-small via FastEmbed) and a local LLM through Ollama
- Text Items (1, 1A, 1C, 3, 7, 7A) in v1; Item 8 tables and XBRL numbers deferred
- Router, retriever and generator as libraries inside one API service

- Rules-first routing with LLM fallback
- Arize Phoenix for tracing

Open

- ☐ Corpus mix: confirm about 50 companies across technology, banking, healthcare, energy and retail for FY2023–FY2025
- ☐ Which 3–4B model to standardize on after a quick routing and answer-quality comparison in E2
- ☐ Access groups: one per sector, or a smaller set such as `tech_coverage`, `financials_coverage`, `admin`
- ☐ Start date for week 1